

Module 4 - Stack, Queue, Decorators

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology

premanand.s@vit.ac.in

September 29, 2025

- A decorator is a callable that takes a function and returns a function.

Decorator - Analogy

- Think of a plain ice-cream (a function).
- A decorator is like adding a topping (chocolate syrup, sprinkles) around the ice-cream without changing the ice-cream itself.
- You still have the same ice-cream, but now it has extra behavior/appearance.

Normal Function

A function is just a block of reusable code.

```
def double(x):  
    return x * 2  
  
print(double(5))  # 10
```

Function inside another function

A function can live inside another.

```
def outer():  
    def inner():  
        return "I am inner!"  
    return inner()  
  
print(outer())
```

Function returning another function

Instead of running the inner function,
the outer can return the function itself.

```
def outer():  
    def inner():  
        return "Hello from inner"  
    return inner # notice no parentheses  
  
f = outer()  
print(f())
```

Functions as parameters

Functions can be passed into other functions.

```
def greet(func):  
    print("Hi!")  
    print(func())  
  
def say_hello():  
    return "Hello there!"  
  
greet(say_hello)
```

Wrapper function

We can create a function that wraps another function | adding extra work before and after it runs.

```
def wrapper(func):  
    def inner():  
        print("Before function runs")  
        func()  
        print("After function runs")  
    return inner
```

```
def say_name():  
    print("My name is Prem")
```

```
wrapped = wrapper(say_name)  
wrapped()
```


Decorator

A decorator is just a shortcut for wrapping.

```
def wrapper(func):  
    def inner():  
        print("Before")  
        func()  
        print("After")  
    return inner
```

```
@wrapper  
def say_hi():  
    print("Hi there!")  
  
say_hi()
```

Decorator with arguments

Sometimes, functions take inputs.
The wrapper should allow that too.

```
def wrapper(func):  
    def inner(*args, **kwargs):  
        print("Start")  
        result = func(*args, **kwargs)  
        print("End")  
        return result  
    return inner  
  
@wrapper  
def add(a, b):  
    return a + b  
  
print(add(2, 3))
```

Multiple Decorators

We can layer decorators like stacking clothes
| jacket over shirt over t-shirt.

```
def star(func):  
    def inner():  
        print("*****")  
        func()  
        print("*****")  
    return inner
```

```
def smile(func):  
    def inner():  
        print(":)")  
        func()  
        print(":)")  
    return inner
```

Multiple Decorators

```
@star
@smile
def say_word():
    print("Python")

say_word()
```

Decorators with arguments

We can give inputs to the decorator itself.

```
def repeat(n):  
    def decorator(func):  
        def inner(*args, **kwargs):  
            for _ in range(n):  
                func(*args, **kwargs)  
        return inner  
    return decorator
```

```
@repeat(3)  
def greet():  
    print("Hello")
```

```
greet()
```

- A linear data structure that follows the LIFO (Last In, First Out) rule.
- Think: Stack of plates $\rightarrow$ you add/remove from the top only.

Basic Operations

- Push → Add an element to the top.
- Pop → Remove the top element.
- Peek/Top → Get the element on top without removing it.
- isEmpty → Check if stack has no elements.
- Size → Number of elements in the stack.

Stack Operation: Push using append()

1. `append()` → Push

`list.append(x)` adds an element `x` at the end of the list.

When using a list as a stack,
the end of the list is considered the top.

So, `append` = Push operation in stack.

```
stack = []  
stack.append(10)  # Push 10  
stack.append(20)  # Push 20  
print("Stack:", stack)  # [10, 20]
```


Stack Operation: Pop

2. Pop Operation

Removes and returns the top element.

If stack is empty → Stack Underflow.

Otherwise → Remove last inserted element.

Example:

```
stack = [10, 20, 30]
```

```
# pop operation
```

```
top_element = stack.pop()
```

```
print("Popped element:", top_element)
```

```
print("Stack after pop:", stack)
```

Stack Operation: Peek / Top

3. Peek / Top Operation

Looks at the top element without removing it.

If stack is empty → No element to peek.

Otherwise → Show last inserted element.

Example:

```
stack = [10, 20, 30]
```

```
# peek operation
```

```
top_element = stack[-1]
```

```
print("Top element is:", top_element)
```

Stack Operation: isEmpty

4. isEmpty Operation

Checks whether the stack has any elements.

Example:

```
stack = []

if not stack:
    print("Stack is empty")
else:
    print("Stack is not empty")
```

Stack Operation: isFull

5. isFull Operation

It checks if stack is full.
(Python list is dynamic, but we can simulate max size.)

Example:

```
stack = []  
MAX_SIZE = 3  
  
def isFull(stack):  
    return len(stack) == MAX_SIZE  
  
stack.append(10)  
stack.append(20)  
stack.append(30)
```

Stack Using queue.LifoQueue

```
from queue import LifoQueue

stack = LifoQueue(maxsize=3)

stack.put(1)
stack.put(2)
stack.put(3)

print("Full?", stack.full())    # True
print("Size:", stack.qsize())   # 3

print("Popped:", stack.get())    # 3
```

Stack Overflow

Stack Overflow occurs when we try to push an element into a stack that is already full (fixed-size stack).

```
stack = []
MAX_SIZE = 3

def isFull(stack):
    return len(stack) == MAX_SIZE

for i in range(1, 5):
    if isFull(stack):
        print("Stack Overflow! Cannot push", i)
    else:
        stack.append(i)
        print("Pushed:", i)
print("Final Stack:", stack)
```

Stack Underflow

Stack Underflow occurs when we try to pop an element from an empty stack.

```
stack = []

if not stack:
    print("Stack Underflow! No element to pop.")
else:
    top_element = stack.pop()
    print("Popped element:", top_element)
```

Application:

- Undo/Redo operations in editors.
- Browser history navigation.
- Expression evaluation (infix $\rightarrow$ postfix).
- Syntax parsing (e.g., matching parentheses).
- DFS (Depth-First Search) in graphs.

- **Push Pop Simulation:** Create a stack using Python list. Push elements [10, 20, 30] and pop the top element, printing the stack after each operation.
- **Peek / Top Element:** Implement a function to return the top element without removing it. Handle empty stack.
- **Check Empty:** Write a program to check whether a stack is empty or not.

- **Stack Overflow / Underflow Handling:** Simulate a fixed-size stack. Handle push when full (overflow) and pop when empty (underflow).
- **Reverse a List Using Stack:** Input: [1,2,3,4,5], Output: [5,4,3,2,1] using a stack.
- **Balanced Parentheses:** Given a string containing (), , [], check if it's balanced using a stack.

- **Next Greater Element:** For a list of integers, print the next greater element for each element. Example: $[4,5,2,25] \rightarrow [5,25,25,-1]$
- **Evaluate Postfix Expression:** Input: $["2","3","*","5","+"] \rightarrow$ Output: 11. Use a stack to evaluate postfix expressions.
- **Implement Two Stacks in One Array:** Use a single Python list to implement two stacks efficiently. Avoid collisions.
- **Largest Rectangle in Histogram:** Given heights of bars, find the largest rectangular area using a stack-based $O(n)$ solution.

- A linear data structure following FIFO (First In, First Out).
- Imagine a line at a ticket counter:
 - First person in line → served first.
 - New people join at the rear (back).
 - People leave from the front.

Queue Operations Overview

- **Enqueue (Insert):** Add element at the rear.
- **Dequeue (Delete):** Remove element from the front.
- **Peek/Front:** View element at the front without removing.
- **isEmpty():** Check if queue is empty.
- **isFull():** Check if queue is full (when MAX_SIZE fixed).
- **Display:** Show elements front $\rightarrow$ rear.

Enqueue and Dequeue

```
# Enqueue (Insert)  
queue.append(item)
```

```
# Dequeue (Delete)  
queue.pop(0)
```

Peek and isEmpty

```
# Peek (Front element)  
queue[0]
```

```
# Check if empty  
len(queue) == 0
```

isFull and Display

```
# Check if full
len(queue) == MAX_SIZE

# Display (front -> rear)
print(queue)
```


Errors in Queue

Overflow

- Occurs when trying to **enqueue (insert)** into a **full queue**.
- Example: Queue with $\text{MAX_SIZE} = 5$ already has 5 elements.

Underflow

- Occurs when trying to **dequeue (delete)** from an **empty queue**.
- Example: Queue is empty, but deletion is attempted.

Summary

- **Overflow** $\Rightarrow$ Queue full, insertion not possible.
- **Underflow** $\Rightarrow$ Queue empty, deletion not possible.

- What is the difference between a stack and a queue?
- In a queue, where are new elements inserted and from where are they removed?
- What is the meaning of **overflow** and **underflow** in a queue?
- Which queue operation corresponds to:
 - `queue.append(x)`
 - `queue.pop(0)`

- Queue = [10, 20, 30, 40]
 - After enqueue(50) → ?
 - After dequeue() → ?
- If MAX_SIZE = 3 and queue = [5, 7, 9], what happens if we do enqueue(11)?
- If the queue is empty and we try dequeue(), what happens?

- How would you simulate a ticket counter line using a queue in Python?
- Why are queues used in CPU scheduling and printer task management?
- In real life, is an ATM withdrawal line a stack or a queue? Why?

- Write a Python program to implement a **menu-driven queue** with operations:
 - Enqueue, Dequeue, Peek, Display
- How would you implement a **circular queue** using a list? Why is it more efficient than a normal queue?
- Differentiate between:
 - Simple Queue
 - Circular Queue
 - Priority Queue
 - Deque
- Implement a queue using two stacks.